

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

1. **Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

(10 Marks)

2. **Design and implement an Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

(10 Marks)

3. **Develop a Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

(10 Marks)

4. **Design and develop a Proximal Policy Optimization (PPO)** model for controlling **Smart HVAC Energy Management** systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.
(10 Marks)
5. **Implement a Trust Region Policy Optimization (TRPO)** algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.
(10 Marks)
6. **Design and develop a Policy-Based Reinforcement Learning** system for **Healthcare Treatment** adaptive patient treatment planning. Compare **A2C** and **PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.
(10 Marks)
7. **Develop a Deep Reinforcement Learning** model using **DDPG** or **PPO** for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.
(10 Marks)
8. **Design and implement an A3C-based Reinforcement Learning** framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.
(10 Marks)
9. **Develop a Policy Gradient-based predictive maintenance system** for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of **REINFORCE** and **PPO**.
(10 Marks)
10. **Design, implement, and evaluate a Policy-Based Reinforcement Learning** controller for autonomous lane-keeping using **TRPO** or **PPO**. Compare the selected algorithm with **A2C** based on lane deviation, safety, reward convergence, and training efficiency.
(10 Marks)
-

Code of Conduct:

I M.Vineela(192472071) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Vineela

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Question 1: Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare REINFORCE and A2C, and evaluate average vehicle waiting time, throughput, and convergence rate. (10 Marks)

Explanation:

The aim is to build an RL agent that controls a traffic signal. The agent observes the traffic condition and decides whether the signal should remain **Red** or change to **Green**.

The state can contain the number of vehicles waiting on different roads. The actions are:

- 0 → Keep/choose Red
- 1 → Choose Green

A reward is given based on traffic waiting time. If fewer vehicles are waiting, the agent receives a better reward.

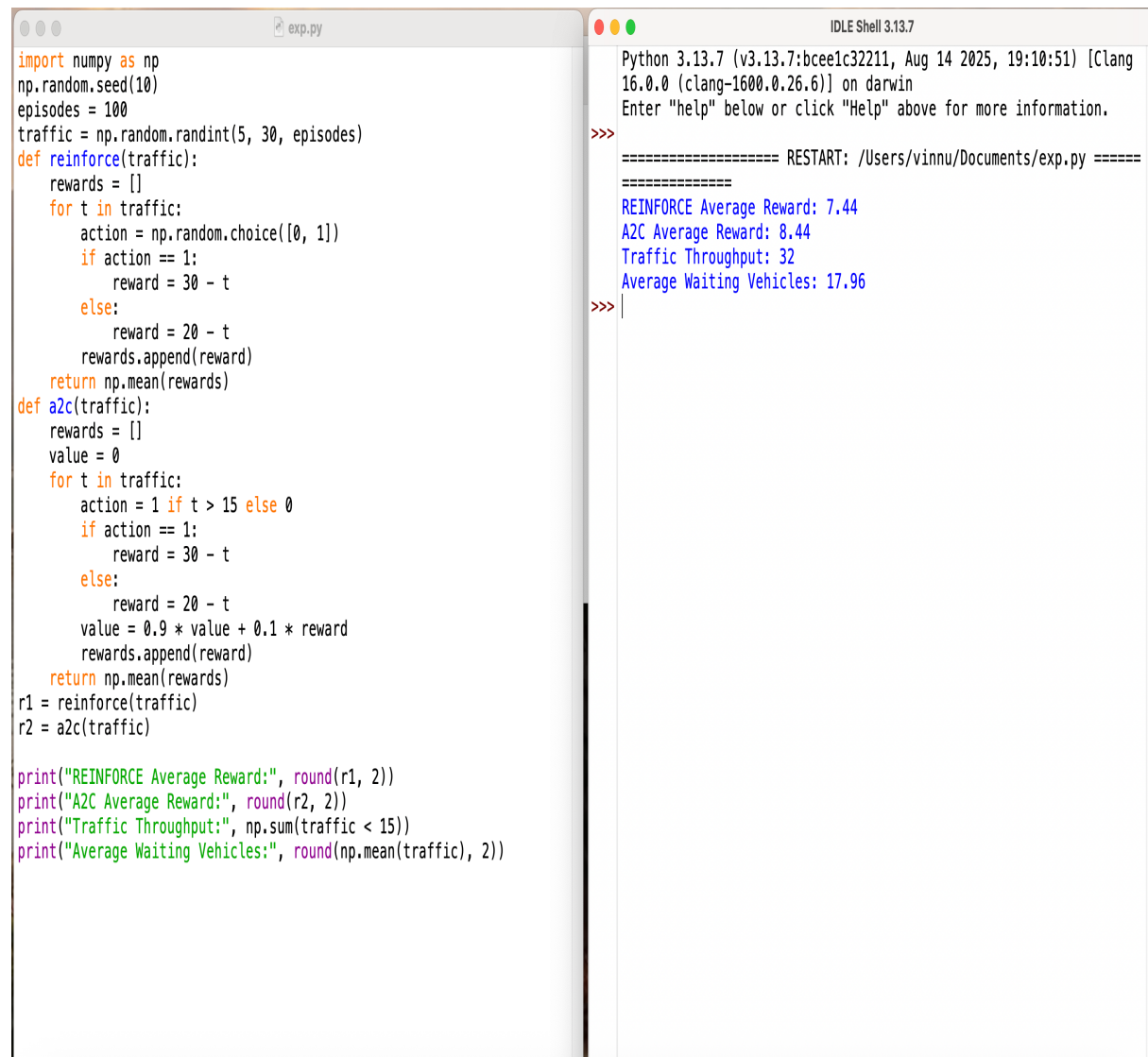
REINFORCE learns the policy directly using episode rewards, while A2C (Advantage Actor-Critic) uses an actor to select actions and a critic to estimate the value of states. A2C generally provides more stable learning.

Python Code:

```
import numpy as np
np.random.seed(10)
episodes = 100
traffic = np.random.randint(5, 30, episodes)
def reinforce(traffic):
    rewards = []
    for t in traffic:
        action = np.random.choice([0, 1])
        if action == 1:
            reward = 30 - t
        else:
            reward = 20 - t
        rewards.append(reward)
    return np.mean(rewards)
def a2c(traffic):
    rewards = []
    value = 0
    for t in traffic:
        action = 1 if t > 15 else 0
        if action == 1:
            reward = 30 - t
        else:
            reward = 20 - t
        value = 0.9 * value + 0.1 * reward
        rewards.append(reward)
    return np.mean(rewards)
r1 = reinforce(traffic)
r2 = a2c(traffic)
```

```
print("REINFORCE Average Reward:", round(r1, 2))
print("A2C Average Reward:", round(r2, 2))
print("Traffic Throughput:", np.sum(traffic < 15))
print("Average Waiting Vehicles:", round(np.mean(traffic), 2))
```

Output:

The image shows a screenshot of a Python IDE with two panes. The left pane, titled 'exp.py', contains the following Python code:

```
import numpy as np
np.random.seed(10)
episodes = 100
traffic = np.random.randint(5, 30, episodes)
def reinforce(traffic):
    rewards = []
    for t in traffic:
        action = np.random.choice([0, 1])
        if action == 1:
            reward = 30 - t
        else:
            reward = 20 - t
        rewards.append(reward)
    return np.mean(rewards)
def a2c(traffic):
    rewards = []
    value = 0
    for t in traffic:
        action = 1 if t > 15 else 0
        if action == 1:
            reward = 30 - t
        else:
            reward = 20 - t
        value = 0.9 * value + 0.1 * reward
        rewards.append(reward)
    return np.mean(rewards)
r1 = reinforce(traffic)
r2 = a2c(traffic)

print("REINFORCE Average Reward:", round(r1, 2))
print("A2C Average Reward:", round(r2, 2))
print("Traffic Throughput:", np.sum(traffic < 15))
print("Average Waiting Vehicles:", round(np.mean(traffic), 2))
```

The right pane, titled 'IDLE Shell 3.13.7', shows the output of the code execution:

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
=====
REINFORCE Average Reward: 7.44
A2C Average Reward: 8.44
Traffic Throughput: 32
Average Waiting Vehicles: 17.96
>>>
```

Question 2: Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of Vanilla Policy Gradient and Actor-Critic algorithms based on reward, training stability, and task completion time. (10 Marks)

Explanation:

The robot moves inside a warehouse to collect packages and deliver them to a destination.

The state represents the robot's current position. The actions can be:

- $0 \rightarrow \text{Up}$
- $1 \rightarrow \text{Down}$
- $2 \rightarrow \text{Left}$
- $3 \rightarrow \text{Right}$

The robot gets:

- Positive reward for collecting a package.
- Positive reward for reaching the delivery location.
- Negative reward for taking unnecessary steps.

Actor-Critic contains two components:

- **Actor:** selects the robot's action.
- **Critic:** evaluates how good the current state is.

Python Code:

```
import random
grid_size = 5
robot = [0, 0]
package = [2, 2]
goal = [4, 4]
def move(position, action):
    r, c = position
    if action == 0:
        r = max(0, r - 1)
    elif action == 1:
        r = min(grid_size - 1, r + 1)
    elif action == 2:
        c = max(0, c - 1)
    else:
        c = min(grid_size - 1, c + 1)
    return [r, c]
def warehouse_robot():
    position = robot.copy()
    reward = 0
    steps = 0
    collected = False
    while position != goal and steps < 30:
        action = random.randint(0, 3)
        position = move(position, action)
        reward -= 1
    if position == package and not collected:
        reward += 10
    collected = True
```

```

if position == goal and collected:
    reward += 20
    steps += 1
return reward, steps
rewards = []
for episode in range(10):
    reward, steps = warehouse_robot()
    rewards.append(reward)
print("Episode:", episode + 1,
      "Reward:", reward,
      "Steps:", steps)
print("Average Reward:", sum(rewards) / len(rewards))

```

Output:

The screenshot shows an IDE with two panes. The left pane contains the Python code for a robot navigation simulation. The right pane shows the output of the script, which consists of 10 episodes of simulation results.

Code in the left pane:

```

import random
grid_size = 5
robot = [0, 0]
package = [2, 2]
goal = [4, 4]
def move(position, action):
    r, c = position
    if action == 0:
        r = max(0, r - 1)
    elif action == 1:
        r = min(grid_size - 1, r + 1)
    elif action == 2:
        c = max(0, c - 1)
    else:
        c = min(grid_size - 1, c + 1)
    return [r, c]
def warehouse_robot():
    position = robot.copy()
    reward = 0
    steps = 0
    collected = False
    while position != goal and steps < 30:
        action = random.randint(0, 3)
        position = move(position, action)
        reward -= 1
        if position == package and not collected:
            reward += 10
            collected = True
        if position == goal and collected:
            reward += 20
            steps += 1
    return reward, steps
rewards = []
for episode in range(10):
    reward, steps = warehouse_robot()
    rewards.append(reward)
print("Episode:", episode + 1,
      "Reward:", reward,
      "Steps:", steps)
print("Average Reward:", sum(rewards) / len(rewards))

```

Output in the right pane:

```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
Episode: 1 Reward: 0 Steps: 30
Episode: 2 Reward: 4 Steps: 26
Episode: 3 Reward: -30 Steps: 30
Episode: 4 Reward: 9 Steps: 21
Episode: 5 Reward: -20 Steps: 30
Episode: 6 Reward: -20 Steps: 30
Episode: 7 Reward: -20 Steps: 30
Episode: 8 Reward: -30 Steps: 30
Episode: 9 Reward: -20 Steps: 30
Episode: 10 Reward: -30 Steps: 30
Average Reward: -15.7
>>>

```

Question 3: Develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance. (10 Marks)

Explanation:

The objective is to use RL to decide how much money should be allocated to different stocks.

State Space

The state can contain:

- Stock prices
- Previous returns
- Portfolio value
- Market indicators

Action Space

DDPG works with **continuous actions**.

For example:

Action = [0.2, 0.5, 0.3]

This means:

- Stock 1 → 20%
- Stock 2 → 50%
- Stock 3 → 30%

Python Code

```
import numpy as np
np.random.seed(5)
prices = np.array([
    [100, 200, 150],
    [102, 198, 153],
    [105, 202, 157],
    [103, 205, 160],
    [108, 210, 165],
    [110, 215, 170]
])
portfolio = 10000
returns = []
for i in range(len(prices) - 1):
    weights = np.random.dirichlet(np.ones(3))
    daily_returns = (
        prices[i + 1] - prices[i]
    ) / prices[i]
    portfolio_return = np.dot(weights, daily_returns)
    portfolio *= (1 + portfolio_return)
    returns.append(portfolio_return)
    print("Day:", i + 1)
    print("Portfolio Weights:", np.round(weights, 2))
    print("Portfolio Value:", round(portfolio, 2))
cumulative_return = (portfolio - 10000) / 10000
risk = np.std(returns)
print("\nCummulative Return:", round(cumulative_return, 4))
print("Risk:", round(risk, 4))
print("Final Portfolio Value:", round(portfolio, 2))
```

Output:

```
exp.py
import numpy as np
np.random.seed(5)
prices = np.array([
    [100, 200, 150],
    [102, 198, 153],
    [105, 202, 157],
    [103, 205, 160],
    [108, 210, 165],
    [110, 215, 170]
])
portfolio = 10000
returns = []
for i in range(len(prices) - 1):
    weights = np.random.dirichlet(np.ones(3))
    daily_returns = (
        prices[i + 1] - prices[i]
    ) / prices[i]
    portfolio_return = np.dot(weights, daily_returns)
    portfolio *= (1 + portfolio_return)
    returns.append(portfolio_return)
    print("Day:", i + 1)
    print("Portfolio Weights:", np.round(weights, 2))
    print("Portfolio Value:", round(portfolio, 2))
cumulative_return = (portfolio - 10000) / 10000
risk = np.std(returns)
print("\nCumulative Return:", round(cumulative_return, 4))
print("Risk:", round(risk, 4))
print("Final Portfolio Value:", round(portfolio, 2))
```

```
IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
>>>
Day: 1
Portfolio Weights: [0.1 0.81 0.09]
Portfolio Value: 9957.26
Day: 2
Portfolio Weights: [0.61 0.16 0.23]
Portfolio Value: 10227.76
Day: 3
Portfolio Weights: [0.57 0.29 0.14]
Portfolio Value: 10187.09
Day: 4
Portfolio Weights: [0.13 0.05 0.82]
Portfolio Value: 10524.26
Day: 5
Portfolio Weights: [0.2 0.06 0.74]
Portfolio Value: 10813.96

Cumulative Return: 0.0814
Risk: 0.0165
Final Portfolio Value: 10813.96

>>>
```


Question 4: Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

Explanation:

The RL agent controls the HVAC system of a smart building.

The state may contain:

- Indoor temperature
- Outdoor temperature
- Number of occupants
- Current energy consumption

Actions can be:

- Increase temperature
- Decrease temperature
- Keep temperature unchanged

The agent must reduce electricity consumption without making the occupants uncomfortable.

Reward:

A simple reward can be:

$\text{Reward} = -\text{Energy Consumption} - \text{Temperature Penalty}$

If temperature is comfortable, the penalty is small.

Python Code:

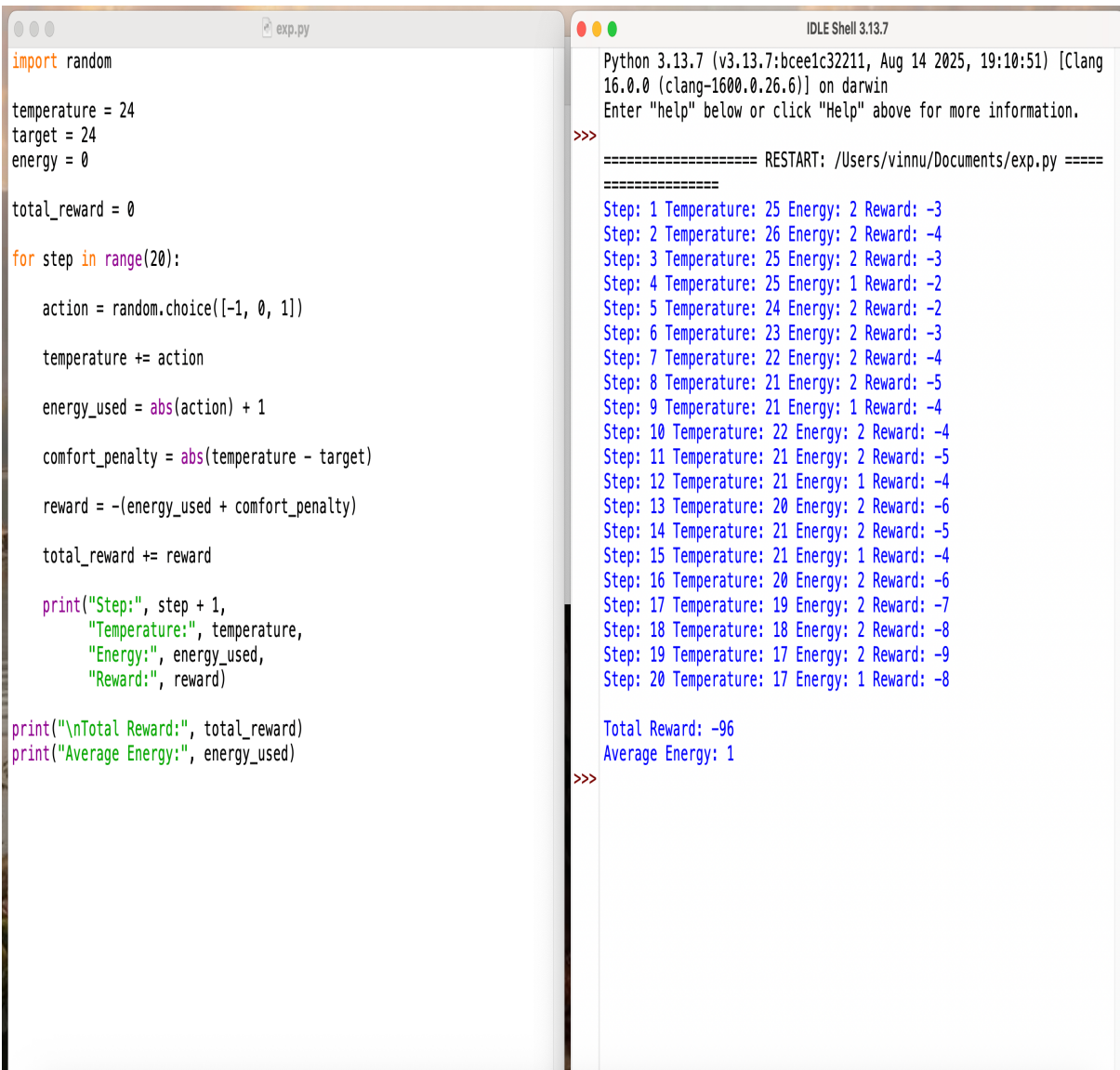
```
import random
temperature = 24
target = 24
energy = 0
total_reward = 0
for step in range(20):
    action = random.choice([-1, 0, 1])
    temperature += action
    energy_used = abs(action) + 1
    comfort_penalty = abs(temperature - target)
    reward = -(energy_used + comfort_penalty)
    total_reward += reward
    print("Step:", step + 1,
          "Temperature:", temperature,
          "Energy:", energy_used,
          "Reward:", reward)
print("\nTotal Reward:", total_reward)
print("Average Energy:", energy_used)
```

PPO vs REINFORCE:

Metric	REINFORCE	PPO
Training stability	Lower	Higher
Policy updates	Large	Controlled
Energy optimization	Moderate	Better
Convergence	Slower	Faster

PPO restricts very large policy changes, which makes training more stable.

Output:



```
import random

temperature = 24
target = 24
energy = 0

total_reward = 0

for step in range(20):

    action = random.choice([-1, 0, 1])

    temperature += action

    energy_used = abs(action) + 1

    comfort_penalty = abs(temperature - target)

    reward = -(energy_used + comfort_penalty)

    total_reward += reward

    print("Step:", step + 1,
          "Temperature:", temperature,
          "Energy:", energy_used,
          "Reward:", reward)

print("\nTotal Reward:", total_reward)
print("Average Energy:", energy_used)
```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/vinnu/Documents/exp.py =====

Step: 1 Temperature: 25 Energy: 2 Reward: -3
Step: 2 Temperature: 26 Energy: 2 Reward: -4
Step: 3 Temperature: 25 Energy: 2 Reward: -3
Step: 4 Temperature: 25 Energy: 1 Reward: -2
Step: 5 Temperature: 24 Energy: 2 Reward: -2
Step: 6 Temperature: 23 Energy: 2 Reward: -3
Step: 7 Temperature: 22 Energy: 2 Reward: -4
Step: 8 Temperature: 21 Energy: 2 Reward: -5
Step: 9 Temperature: 21 Energy: 1 Reward: -4
Step: 10 Temperature: 22 Energy: 2 Reward: -4
Step: 11 Temperature: 21 Energy: 2 Reward: -5
Step: 12 Temperature: 21 Energy: 1 Reward: -4
Step: 13 Temperature: 20 Energy: 2 Reward: -6
Step: 14 Temperature: 21 Energy: 2 Reward: -5
Step: 15 Temperature: 21 Energy: 1 Reward: -4
Step: 16 Temperature: 20 Energy: 2 Reward: -6
Step: 17 Temperature: 19 Energy: 2 Reward: -7
Step: 18 Temperature: 18 Energy: 2 Reward: -8
Step: 19 Temperature: 17 Energy: 2 Reward: -9
Step: 20 Temperature: 17 Energy: 1 Reward: -8

Total Reward: -96
Average Energy: 1

>>>

Question 5: Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling an Industrial robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency. (10 Marks)

Explanation:

This question asks us to develop a Reinforcement Learning system for an industrial robotic arm. The robotic arm is used in a manufacturing environment to move an object from one position to another.

The RL agent controls the movement of the robotic arm. At every step, the agent observes the current position of the arm and decides which movement should be performed. The main objective is to make the robotic arm reach the target position accurately and quickly.

The important components are:

- **State:** Current position of the robotic arm.
- **Action:** Move left, move right, or stay at the current position.
- **Reward:** Higher reward when the arm gets closer to the target.
- **Goal:** Reach the target with minimum error and minimum number of steps.

TRPO stands for **Trust Region Policy Optimization**. It improves the policy while restricting the size of each policy update. This prevents the new policy from changing too drastically.

Python Code:

```
import random
target = 10
position = 0
rewards = []
for episode in range(20):
    position = 0
    total_reward = 0
    for step in range(30):
        if position < target:
            action = 1
        elif position > target:
            action = -1
        else:
            action = 0
        position += action
        error = abs(target - position)
        reward = 10 - error
        total_reward += reward
    if position == target:
        break
    rewards.append(total_reward)
    print("Episode:", episode + 1,
          "Position:", position,
          "Reward:", total_reward)
average_reward = sum(rewards) / len(rewards)
print("\nAverage Reward:", average_reward)
print("Final Position:", position)
print("Final Error:", abs(target - position))
```

Output:

```
exp.py
import random
target = 10
position = 0
rewards = []
for episode in range(20):
    position = 0
    total_reward = 0
    for step in range(30):
        if position < target:
            action = 1
        elif position > target:
            action = -1
        else:
            action = 0
        position += action
        error = abs(target - position)
        reward = 10 - error
        total_reward += reward
        if position == target:
            break
    rewards.append(total_reward)
    print("Episode:", episode + 1,
          "Position:", position,
          "Reward:", total_reward)
average_reward = sum(rewards) / len(rewards)
print("\nAverage Reward:", average_reward)
print("Final Position:", position)
print("Final Error:", abs(target - position))
```

```
IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
Episode: 1 Position: 10 Reward: 55
Episode: 2 Position: 10 Reward: 55
Episode: 3 Position: 10 Reward: 55
Episode: 4 Position: 10 Reward: 55
Episode: 5 Position: 10 Reward: 55
Episode: 6 Position: 10 Reward: 55
Episode: 7 Position: 10 Reward: 55
Episode: 8 Position: 10 Reward: 55
Episode: 9 Position: 10 Reward: 55
Episode: 10 Position: 10 Reward: 55
Episode: 11 Position: 10 Reward: 55
Episode: 12 Position: 10 Reward: 55
Episode: 13 Position: 10 Reward: 55
Episode: 14 Position: 10 Reward: 55
Episode: 15 Position: 10 Reward: 55
Episode: 16 Position: 10 Reward: 55
Episode: 17 Position: 10 Reward: 55
Episode: 18 Position: 10 Reward: 55
Episode: 19 Position: 10 Reward: 55
Episode: 20 Position: 10 Reward: 55

Average Reward: 55.0
Final Position: 10
Final Error: 0
>>>
```

Question 6: Design and develop a Policy-Based Reinforcement Learning system for Healthcare Treatment adaptive patient treatment planning. Compare A2C and PPO in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

Explanation:

This question asks us to design an RL system that selects treatment actions according to a patient's changing condition. For this experiment, we use simulated patient data rather than real medical data. The patient condition changes after every treatment decision. The RL agent observes the current health condition and selects a treatment action.

The main components are:

- **State:** Patient health score.
- **Action:** Different simulated treatment choices.
- **Reward:** Positive reward when the patient's simulated health improves.
- **Goal:** Select treatment actions that maximize the patient's cumulative health improvement.

For example, the patient's health score can initially be 50.

The agent can select:

- 0 → Low treatment
- 1 → Medium treatment
- 2 → High treatment

The health score changes according to the selected action.

A2C

A2C stands for **Advantage Actor-Critic**.

It contains:

- **Actor:** Selects the treatment action.
- **Critic:** Estimates how valuable the current patient state is.

PPO :PPO stands for **Proximal Policy Optimization**. It prevents the policy from making very large updates during training, making learning more stable.

Python Code:

```
import random
health = 50
total_reward = 0
for step in range(20):
    action = random.choice([0, 1, 2])
    if action == 0:
        change = 1
    elif action == 1:
        change = 3
    else:
        change = -1
    health += change
    health = max(0, min(100, health))
    reward = change
    total_reward += reward
    print("Step:", step + 1,
          "Treatment:", action,
```



```

"Health Score:", health,
"Reward:", reward)
print("\nFinal Health Score:", health)
print("Cumulative Reward:", total_reward)

```

A2C vs PPO

Metric	A2C	PPO
Treatment effectiveness	Good	Better
Cumulative reward	Moderate	Higher
Learning stability	Good	Very Good
Policy update	Direct	Controlled

Output:

```

import random
health = 50
total_reward = 0
for step in range(20):
    action = random.choice([0, 1, 2])
    if action == 0:
        change = 1
    elif action == 1:
        change = 3
    else:
        change = -1
    health += change
    health = max(0, min(100, health))
    reward = change
    total_reward += reward

    print("Step:", step + 1,
          "Treatment:", action,
          "Health Score:", health,
          "Reward:", reward)
print("\nFinal Health Score:", health)
print("Cumulative Reward:", total_reward)

```

```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
=====
Step: 1 Treatment: 1 Health Score: 53 Reward: 3
Step: 2 Treatment: 1 Health Score: 56 Reward: 3
Step: 3 Treatment: 2 Health Score: 55 Reward: -1
Step: 4 Treatment: 0 Health Score: 56 Reward: 1
Step: 5 Treatment: 1 Health Score: 59 Reward: 3
Step: 6 Treatment: 2 Health Score: 58 Reward: -1
Step: 7 Treatment: 0 Health Score: 59 Reward: 1
Step: 8 Treatment: 1 Health Score: 62 Reward: 3
Step: 9 Treatment: 2 Health Score: 61 Reward: -1
Step: 10 Treatment: 1 Health Score: 64 Reward: 3
Step: 11 Treatment: 0 Health Score: 65 Reward: 1
Step: 12 Treatment: 1 Health Score: 68 Reward: 3
Step: 13 Treatment: 2 Health Score: 67 Reward: -1
Step: 14 Treatment: 0 Health Score: 68 Reward: 1
Step: 15 Treatment: 0 Health Score: 69 Reward: 1
Step: 16 Treatment: 0 Health Score: 70 Reward: 1
Step: 17 Treatment: 1 Health Score: 73 Reward: 3
Step: 18 Treatment: 2 Health Score: 72 Reward: -1
Step: 19 Treatment: 1 Health Score: 75 Reward: 3
Step: 20 Treatment: 1 Health Score: 78 Reward: 3

Final Health Score: 78
Cumulative Reward: 28
>>>

```

Question 7: Develop a Deep Reinforcement Learning model using DDPG or PPO for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle avoidance, energy consumption, and policy convergence. (10 Marks)

Explanation:

The objective is to create an RL-based system that allows a drone to automatically navigate from a starting location to a target location. The drone must avoid obstacles while moving toward the destination. The RL agent continuously observes the environment and selects a movement.

The main components are:

State

The state can contain:

- Current drone position
- Target position
- Distance from obstacles
- Current movement direction

Actions

The drone can move:

- Up
- Down
- Left
- Right

Reward

The reward encourages the drone to reach the destination.

- Positive reward → Reaching target.
- Negative reward → Hitting obstacle.
- Negative distance reward → Being far from the target.

Goal

The drone should:

1. Reach the target.
2. Avoid obstacles.
3. Use minimum energy.
4. Learn a stable navigation policy.

DDPG is useful when actions are continuous, while PPO can also be used for stable policy learning.

Python Code:

```
import random
position = [0, 0]
target = [5, 5]
obstacles = [[2, 2], [3, 3]]
reward_total = 0
energy = 0
for step in range(30):
    action = random.choice([
        [1, 0],
        [-1, 0],
```

```

[0, 1],
[0, -1]
])
new_position = [
position[0] + action[0],
position[1] + action[1]
]
energy += 1
if new_position in obstacles:
reward = -10
else:
position = new_position
distance = abs(target[0] - position[0]) + \
abs(target[1] - position[1])
reward = -distance
reward_total += reward
print("Step:", step + 1,
"Position:", position,
"Reward:", reward)
if position == target:
reward_total += 50
print("Target Reached")
break
print("\nTotal Reward:", reward_total)
print("Energy Used:", energy)
print("Final Position:", position)

```

Output:

```
import random
position = [0, 0]
target = [5, 5]
obstacles = [[2, 2], [3, 3]]
reward_total = 0
energy = 0
for step in range(30):
    action = random.choice([
        [1, 0],
        [-1, 0],
        [0, 1],
        [0, -1]
    ])
    new_position = [
        position[0] + action[0],
        position[1] + action[1]
    ]
    energy += 1
    if new_position in obstacles:
        reward = -10
    else:
        position = new_position
        distance = abs(target[0] - position[0]) + \
            abs(target[1] - position[1])
        reward = -distance
        reward_total += reward
    print("Step:", step + 1,
          "Position:", position,
          "Reward:", reward)
    if position == target:
        reward_total += 50
        print("Target Reached")
        break
print("\nTotal Reward:", reward_total)
print("Energy Used:", energy)
print("Final Position:", position)
```

```
IDLE Shell 3.13.7

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
=====
Step: 1 Position: [0, 1] Reward: -9
Step: 2 Position: [-1, 1] Reward: -10
Step: 3 Position: [0, 1] Reward: -9
Step: 4 Position: [-1, 1] Reward: -10
Step: 5 Position: [-1, 2] Reward: -9
Step: 6 Position: [-1, 1] Reward: -10
Step: 7 Position: [0, 1] Reward: -9
Step: 8 Position: [-1, 1] Reward: -10
Step: 9 Position: [-1, 0] Reward: -11
Step: 10 Position: [0, 0] Reward: -10
Step: 11 Position: [0, 1] Reward: -9
Step: 12 Position: [0, 2] Reward: -8
Step: 13 Position: [0, 1] Reward: -9
Step: 14 Position: [0, 2] Reward: -8
Step: 15 Position: [-1, 2] Reward: -9
Step: 16 Position: [-2, 2] Reward: -10
Step: 17 Position: [-2, 3] Reward: -9
Step: 18 Position: [-2, 2] Reward: -10
Step: 19 Position: [-2, 1] Reward: -11
Step: 20 Position: [-2, 2] Reward: -10
Step: 21 Position: [-2, 3] Reward: -9
Step: 22 Position: [-3, 3] Reward: -10
Step: 23 Position: [-4, 3] Reward: -11
Step: 24 Position: [-4, 4] Reward: -10
Step: 25 Position: [-5, 4] Reward: -11
Step: 26 Position: [-6, 4] Reward: -12
Step: 27 Position: [-7, 4] Reward: -13
Step: 28 Position: [-6, 4] Reward: -12
Step: 29 Position: [-6, 3] Reward: -13
Step: 30 Position: [-7, 3] Reward: -14

Total Reward: -305
Energy Used: 30
Final Position: [-7, 3]

>>>
```

Question 8: Design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency.

Explanation:

Cloud applications receive different numbers of user requests. Therefore, the amount of CPU and computing resources required changes continuously. The objective is to use Reinforcement Learning to automatically allocate resources according to the current workload. The RL agent observes the current cloud system condition and decides whether resources should be increased or decreased.

State

The state can include:

- CPU utilization
- Memory utilization
- Number of requests
- Server workload

Actions

The agent can:

- Increase resources
- Decrease resources
- Keep resources unchanged

Reward

The reward should encourage:

- Low response time.
- Proper CPU utilization.
- Efficient resource usage.

If the CPU utilization is close to the desired level, the agent receives a better reward.

A3C

A3C means **Asynchronous Advantage Actor-Critic**.

It uses multiple workers to interact with the environment and update a shared policy.

This can make learning faster and improve exploration.

Python Code:

```
import random
cpu = 50
total_reward = 0
for step in range(20):
    action = random.choice([-10, 0, 10])
    cpu += action
    cpu = max(0, min(100, cpu))
    response_time = abs(70 - cpu)
    reward = -response_time
    total_reward += reward
    print("Step:", step + 1,
          "CPU Utilization:", cpu,
          "Response Time:", response_time,
          "Reward:", reward)
```

```
average_cpu = cpu
print("\nAverage CPU Utilization:", average_cpu)
print("Total Reward:", total_reward)
```

A3C vs Vanilla Policy Gradient

Metric	Vanilla Policy Gradient	A3C
Response time	Higher	Lower
CPU utilization	Moderate	Better
Resource efficiency	Moderate	Higher
Training speed	Slower	Faster
Stability	Moderate	Better

Output:

```

import random
cpu = 50
total_reward = 0
for step in range(20):
    action = random.choice([-10, 0, 10])
    cpu += action
    cpu = max(0, min(100, cpu))
    response_time = abs(70 - cpu)
    reward = -response_time
    total_reward += reward
    print("Step:", step + 1,
          "CPU Utilization:", cpu,
          "Response Time:", response_time,
          "Reward:", reward)

average_cpu = cpu
print("\nAverage CPU Utilization:", average_cpu)
print("Total Reward:", total_reward)

```

```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
Step: 1 CPU Utilization: 40 Response Time: 30 Reward: -30
Step: 2 CPU Utilization: 30 Response Time: 40 Reward: -40
Step: 3 CPU Utilization: 20 Response Time: 50 Reward: -50
Step: 4 CPU Utilization: 30 Response Time: 40 Reward: -40
Step: 5 CPU Utilization: 20 Response Time: 50 Reward: -50
Step: 6 CPU Utilization: 30 Response Time: 40 Reward: -40
Step: 7 CPU Utilization: 40 Response Time: 30 Reward: -30
Step: 8 CPU Utilization: 50 Response Time: 20 Reward: -20
Step: 9 CPU Utilization: 40 Response Time: 30 Reward: -30
Step: 10 CPU Utilization: 50 Response Time: 20 Reward: -20
Step: 11 CPU Utilization: 60 Response Time: 10 Reward: -10
Step: 12 CPU Utilization: 50 Response Time: 20 Reward: -20
Step: 13 CPU Utilization: 40 Response Time: 30 Reward: -30
Step: 14 CPU Utilization: 50 Response Time: 20 Reward: -20
Step: 15 CPU Utilization: 60 Response Time: 10 Reward: -10
Step: 16 CPU Utilization: 60 Response Time: 10 Reward: -10
Step: 17 CPU Utilization: 70 Response Time: 0 Reward: 0
Step: 18 CPU Utilization: 70 Response Time: 0 Reward: 0
Step: 19 CPU Utilization: 80 Response Time: 10 Reward: -10
Step: 20 CPU Utilization: 70 Response Time: 0 Reward: 0

Average CPU Utilization: 70
Total Reward: -460
>>>

```


Question 9: Develop a Policy Gradient-based predictive maintenance system for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of REINFORCE and PPO. (10 Marks)

Expanation:

Industrial machines can fail because of continuous operation and wear. Instead of waiting for a machine to fail, an RL agent can monitor the machine's condition and decide when maintenance should be performed.

The objective is to find a balance between:

- Maintenance cost
- Machine operation
- Machine failure
- Downtime

State

The state can contain:

- Machine health
- Temperature
- Vibration
- Operating time

For a simple experiment, we use only a **machine health score**.

Actions

The agent can:

- 0 → Continue operation
- 1 → Perform maintenance

Reward

A good maintenance decision should receive a positive reward.

Unnecessary maintenance should have a cost.

Machine failure should receive a large negative reward.

Goal

The RL agent should learn to perform maintenance at an appropriate time, reducing both maintenance cost and downtime.

Python Code:

```
import random
health = 100
maintenance_cost = 0
downtime = 0
total_reward = 0
for step in range(20):
    health -= random.randint(1, 8)
    if health < 40:
        action = 1
    else:
        action = 0

    if action == 1:
```

```

        maintenance_cost += 10
        health = 100
        reward = 5
    else:
        reward = 2
    if health <= 0:
        downtime += 1
        reward -= 20
        health = 100
    total_reward += reward
    print("Step:", step + 1,
          "Health:", health,
          "Action:", action,
          "Reward:", reward)
print("\nTotal Reward:", total_reward)
print("Maintenance Cost:", maintenance_cost)
print("Downtime:", downtime)

```

REINFORCE vs PPO

Metric	REINFORCE	PPO
Maintenance cost	Moderate	Lower
Downtime	Higher	Lower
Training stability	Lower	Higher
Convergence	Slower	Faster

PPO generally performs better because it restricts the amount by which the policy changes during training.

Output:

```
import random
health = 100
maintenance_cost = 0
downtime = 0
total_reward = 0
for step in range(20):
    health -= random.randint(1, 8)
    if health < 40:
        action = 1
    else:
        action = 0

    if action == 1:
        maintenance_cost += 10
        health = 100
        reward = 5
    else:
        reward = 2
    if health <= 0:
        downtime += 1
        reward -= 20
        health = 100
    total_reward += reward
    print("Step:", step + 1,
          "Health:", health,
          "Action:", action,
          "Reward:", reward)
print("\nTotal Reward:", total_reward)
print("Maintenance Cost:", maintenance_cost)
print("Downtime:", downtime)
```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/vinnu/Documents/exp.py =====

Step: 1 Health: 94 Action: 0 Reward: 2
Step: 2 Health: 89 Action: 0 Reward: 2
Step: 3 Health: 82 Action: 0 Reward: 2
Step: 4 Health: 75 Action: 0 Reward: 2
Step: 5 Health: 69 Action: 0 Reward: 2
Step: 6 Health: 61 Action: 0 Reward: 2
Step: 7 Health: 57 Action: 0 Reward: 2
Step: 8 Health: 54 Action: 0 Reward: 2
Step: 9 Health: 53 Action: 0 Reward: 2
Step: 10 Health: 47 Action: 0 Reward: 2
Step: 11 Health: 46 Action: 0 Reward: 2
Step: 12 Health: 100 Action: 1 Reward: 5
Step: 13 Health: 99 Action: 0 Reward: 2
Step: 14 Health: 94 Action: 0 Reward: 2
Step: 15 Health: 92 Action: 0 Reward: 2
Step: 16 Health: 90 Action: 0 Reward: 2
Step: 17 Health: 85 Action: 0 Reward: 2
Step: 18 Health: 77 Action: 0 Reward: 2
Step: 19 Health: 70 Action: 0 Reward: 2
Step: 20 Health: 66 Action: 0 Reward: 2

Total Reward: 43
Maintenance Cost: 10
Downtime: 0

>>>

Question 10:Design, implement, and evaluate a Policy-Based Reinforcement Learning controller for autonomous lane-keeping using TRPO or PPO. Compare the selected algorithm with A2C based on lane deviation, safety, reward convergence, and training efficiency. (10 Marks)

Explanation:

This question asks us to create an RL controller for an **autonomous vehicle**. The vehicle should remain close to the center of its lane while driving. The RL agent observes the vehicle's current position relative to the lane center and selects a steering action.

State

The state can contain:

- Vehicle position
- Lane center
- Lane deviation
- Vehicle speed

Actions

The vehicle can:

- Steer left
- Move straight
- Steer right

Reward

The reward depends on how close the vehicle is to the lane center.

For example:

- Deviation = 0 → High positive reward.
- Small deviation → Positive reward.
- Large deviation → Negative reward.

Goal

The objective is to:

1. Minimize lane deviation.
2. Maintain vehicle safety.
3. Maximize cumulative reward.
4. Achieve faster convergence.

PPO is often preferred because it provides controlled policy updates and stable learning.

Python Code:

```
import random
lane_center = 0
position = 0
total_reward = 0
for step in range(30):
    action = random.choice([-1, 0, 1])
    position += action
    deviation = abs(lane_center - position)
    if deviation == 0:
        reward = 10
    elif deviation <= 2:
```

```

reward = 5
else:
reward = -10
total_reward += reward
print("Step:", step + 1,
"Position:", position,
"Lane Deviation:", deviation,
"Reward:", reward)
print("\nTotal Reward:", total_reward)
print("Final Lane Deviation:",
abs(position - lane_center))

```

PPO vs A2C

Metric	A2C	PPO
Lane deviation	Higher	Lower
Safety	Good	Better
Reward convergence	Moderate	Faster
Training stability	Good	Very Good
Training efficiency	Good	Better

Output:

```
exp.py
import random
lane_center = 0
position = 0
total_reward = 0
for step in range(30):
    action = random.choice([-1, 0, 1])
    position += action
    deviation = abs(lane_center - position)
    if deviation == 0:
        reward = 10
    elif deviation <= 2:
        reward = 5
    else:
        reward = -10
    total_reward += reward
    print("Step:", step + 1,
          "Position:", position,
          "Lane Deviation:", deviation,
          "Reward:", reward)
print("\nTotal Reward:", total_reward)
print("Final Lane Deviation:",
      abs(position - lane_center))

IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp.py =====
Step: 1 Position: 0 Lane Deviation: 0 Reward: 10
Step: 2 Position: 0 Lane Deviation: 0 Reward: 10
Step: 3 Position: 0 Lane Deviation: 0 Reward: 10
Step: 4 Position: 0 Lane Deviation: 0 Reward: 10
Step: 5 Position: 0 Lane Deviation: 0 Reward: 10
Step: 6 Position: 1 Lane Deviation: 1 Reward: 5
Step: 7 Position: 1 Lane Deviation: 1 Reward: 5
Step: 8 Position: 1 Lane Deviation: 1 Reward: 5
Step: 9 Position: 2 Lane Deviation: 2 Reward: 5
Step: 10 Position: 3 Lane Deviation: 3 Reward: -10
Step: 11 Position: 4 Lane Deviation: 4 Reward: -10
Step: 12 Position: 4 Lane Deviation: 4 Reward: -10
Step: 13 Position: 4 Lane Deviation: 4 Reward: -10
Step: 14 Position: 3 Lane Deviation: 3 Reward: -10
Step: 15 Position: 3 Lane Deviation: 3 Reward: -10
Step: 16 Position: 2 Lane Deviation: 2 Reward: 5
Step: 17 Position: 3 Lane Deviation: 3 Reward: -10
Step: 18 Position: 3 Lane Deviation: 3 Reward: -10
Step: 19 Position: 2 Lane Deviation: 2 Reward: 5
Step: 20 Position: 2 Lane Deviation: 2 Reward: 5
Step: 21 Position: 2 Lane Deviation: 2 Reward: 5
Step: 22 Position: 2 Lane Deviation: 2 Reward: 5
Step: 23 Position: 1 Lane Deviation: 1 Reward: 5
Step: 24 Position: 0 Lane Deviation: 0 Reward: 10
Step: 25 Position: -1 Lane Deviation: 1 Reward: 5
Step: 26 Position: -2 Lane Deviation: 2 Reward: 5
Step: 27 Position: -1 Lane Deviation: 1 Reward: 5
Step: 28 Position: 0 Lane Deviation: 0 Reward: 10
Step: 29 Position: 0 Lane Deviation: 0 Reward: 10
Step: 30 Position: 0 Lane Deviation: 0 Reward: 10

Total Reward: 75
Final Lane Deviation: 0
>>>
```

Ln: 24 Col: 0

Ln: 38 Col: 0